Programa:	File Recover 5.0.1.15
Protección:	Armadillo 3.78 + Crippleware
Objetivos:	Desempakar y evitar las limitaciones
URL descarga:	www.pctools.com
Dificultad:	No es difícil
Herramientas:	OllyDbg 1.10 + Import Reconstructor 1.6 + UltraEdit + PEiD 0.93
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos] Fecha: 08-09-2005

Hola a todos..!!

Hoy abordaremos dos cuestiones principalmente que son las que me interesan, por un lado el desempaquetado manual de Armadillo sin CopyMemII y por otro la activación de funciones deshabilitadas, práctica muy extendia en los programas Shareware de hoy en día. Para afrontar el tutorial con garantías no es necesario NADA, absolutamente nada, un poco de práctica no viene mal pero no es imprescindible, cualquiera con conocimientos mínimos lo puede seguir garantizado, se trata de que los que empiezan lo puedan realizar, así que no usaremos ni tecnicsmos ni nada de eso ok? Se hará referencia a las cosas por su nombre común, vamos...

Desempakado de Armadillo 3.78 sin CopyMemII...

Este es un tema que asusta a muchos pero hoy de verdad que se les quitará el respeto por esta protección, es de reconocer que las nuevas protecciones son espantosas (no vale el truco del injerto para el CopyMemII y demás cosas) además de incorporar Nanomites en algunos casos, un trabajo al go engorroso de llevar, está bien cuando nunca los has encontrado, pero una vez te topas con ellos se te quitan las ganas ;-)

Vamos a comenzar como siempre por mirar con qué está comprimido/encriptado el ejecutable, es un paso no obligatorio pero si muy recomendable, veamos:

Armadillo 3.78 -> Silicon Realms Toolworks [Overlay]

Pues bien, nuestro colega el Armadillo, jejeje…, a ver con qué nuevas sorpresas nos depara ya que con nunca se sabe con el Armna, de momento miramos cuantos procesos carga, ejecutamos el programa desde el explorer sin depurar de momento y lo miramos dedse el Olly->File/Attach, con LordPE, etc., yo lo miro con LordPE y veo esto:

Path	PID	ImageBase	ImageSize
c:\archivos de programa\techsmith\snagit 7\...	00000DE0	00400000	00390000
c:\archivos de programa\techsmith\snagit 7\t...	00000BA8	00400000	0000A000
e:\setups\imprec1.6\importrec.exe	00000E30	00400000	0007C000
e:\setups\lordpe\lordpe.exe	00000FA0	00400000	00036000
c:\archivos de programa\file recover 5.0\filer...	00000F7C	00400000	000E0000

un solo proceso, descartamos Nanomites y CopyMem, lo cerramos y ahora sí lo abrimos con Olly, primero veamos si corre, damos RUN y aparece un mensaje de error de S.O. que cuando lo aceptamos se cierra Olly ¿? Está claro que utiliza algún truquito, entonces se me ocurrió que podría estar utilizando las conocidas llamadas a OutputDebug… típicas de esta versión, esto lo resolvemos (si es que es así) poniendo un BP en la api desde el CommandBar ponemos:

Command BP OutputDebugStringA | BP address, string -- Break with condition

damos RUN y al poco Olly para, miramos al stack y vemos:

```
0012CCBC  00AE1F4B  CALL to OutputDebugStringA from 00AE1F45
0012CCC0  0012D5E4  String = "%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%
0012CCC4  0012FF04
```

parece ser que esto es lo que genera el conflicto, nos situamos en el RET de la función y hacemos un ***New origin here*** en él para situar allí el EIP y no se ejecute la API, esto se repite hasta 4 veces y ya carga el programa, bouuuuuff..

Ahora que conseguimos que el programa corra en Olly vamos a dumpearlo, yo os diré el modo en el cual yo lo hago, tal cual secuencialmente, otros tutoriales seguramente diferirán de este aunque el resultado eso sí es el mismo. Reiniciamos Olly, ponemos el BP en OutputDebug… (no lo diré posteriormente pero queda aquí dicho que se hará cada vez que reiniciemos Olly) como

sabemos que para 4 veces en la API, la última vez antes de dar RUN pondremos el BPM On Acces en la primera sección para saber cuando salte al OEP del programa, para aquí:

```
Address   Hex dump    Disassembly                                  Comment
0041FAFD  6A 74       PUSH 74                  OEP
0041FAFF  68 20414200 PUSH FileReco.00424120
0041FB04  E8 BFFDFFFF CALL FileReco.0041F8C8
0041FB09  33DB        XOR EBX,EBX
0041FB0B  895D E0     MOV DWORD PTR SS:[EBP-20],EBX
```

Es un poco raro pero es nuestro OEP no hay duda, lo anotamos:

OEP: 41FAFD

Ya que estamos busquemos la IAT, un poco más abajo tenemos un CALL que va perfecto, lo seguimos Follow in Dump->Memory Address y caemos en ella, buscamos el inicio:

```
Address   Hex dump                                         ASCII
00421000  DA F8 AC 00 9A 22 DA 77 10 24 DA 77 B1 63 DB 77  ┌°¼.Ü"┌w►$┌w▒c▄w
00421010  D4 65 DB 77 E2 68 DB 77 8B 6F DB 77 99 4E DA 77  ╘e▄wôh▄wïo▄wÖN┌w
00421020  14 30 DA 77 3B 20 DA 77 6D 32 DA 77 7B 20 DA 77  ¶0┌w; ┌wm2┌w{ ┌w
00421030  9B 20 DA 77 6F 30 DA 77 ED 30 DA 77 CD 30 DA 77  ¢ ┌wo0┌wÝ0┌w═0┌w
```

Este es, además coincide con el inicio de la primera sección, luego lo anotamos:

Inicio IAT: 421000

Y buscamos el final de la misma manera:

```
Address   Hex dump                                         ASCII
00421560  7E BF AC 00 62 8B B0 76 C0 BF AC 00 17 0E B2 00  ~┐¼.b╗▒v└┐¼.♫♥▓.
00421570  41 07 B2 00 53 07 B2 00 1C 9F B5 00 C6 82 B2 00  A•▓.S•▓.∟ƒµ.Æé▓.
00421580  77 21 B2 00 B6 BF AC 00 00 00 00 00 32 00 00 00  w!▓.Ã┐¼.....2...
00421590  FF C9 9A 3B B8 0B 00 00 00 00 00 00 00 00 00 00  ╠É:╗♂..........
```

No está tan claro pero no tiene complicación verlo, lo anotamos también:

Final IAT: 421588

Siempre le doy un mínimo margen al final, con estos datos sacamos su largo:

Largo = final - inicio: 421588 - 421500 = 588
Longitud IAT: 588

Ya tenemos los datos suficientes para luego ir al Import y reparar la IAT, el siguiente paso será encontrar el salto mágico para que no nos redirija la tabla, ponemos un BPHardware on Memory Write en un puntero redirigido, elegiremos [421000] que es el primero de todos cuyos bytes finales son ACF8DA que anotamos también para saber donde está rellando la tabla final, hay veces en las que parará en varios sitios a escribir en ella y esto diferenciará la vez que nos vale, puesto el BP reiniciamos Olly y damos RUN (el BP de OutputDebug…) para aquí:

```
77C12F43    F3:A5           REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
77C12F45    FF2495 5830C177 JMP DWORD PTR DS:[EDX*4+77C13058]
```

Aqui para tras superar las dos primeras llamadas a OutputDebugStringA, pero no guarda los bytes finales, damos RUN nuevamente y para en esta otra parte:

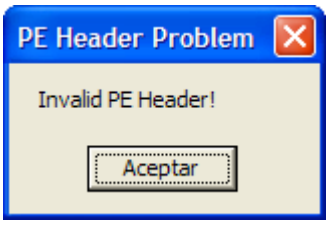
```
00AE902E    8908            MOV DWORD PTR DS:[EAX],ECX          ;Guarda la API buena o mala
00AE9030    8B85 FCC7FFFF   MOV EAX,DWORD PTR SS:[EBP-3804]     ;FileReco.00421000
00AE9036    83C0 04         ADD EAX,4                           ;Apunta a la sgute entrada
00AE9039    8985 FCC7FFFF   MOV DWORD PTR SS:[EBP-3804],EAX
00AE903F  ^ E9 4DFCFFFF     JMP 00AE8C91                        ;Salta al inicio del loop
```

Esta zona sí que es la que buscamos, vemos que guardó el valor 00ACF8DA que era el que nosotros esperábamos, el siguiente paso es buscar el salto mágico, no cuesta mucho encontrarlo, lo que suelo hacer es buscar el call donde pone en EAX la API correcta y mirar los posibles saltos que de producirse superen dicho call, en este caso vemos este:

```
00AE8ED8    83BD 50B9FFFF 0>CMP DWORD PTR SS:[EBP+FFFFB950],0
00AE8EDF    75 42           JNZ SHORT 00AE8F23                  ;Salto mágico..!!
00AE8EE1    0FB785 54B9FFFF MOVZX EAX,WORD PTR SS:[EBP+FFFFB954]
00AE8EE8    85C0            TEST EAX,EAX
00AE8EEA    74 0F           JE SHORT 00AE8EFB
00AE8EEC    0FB785 54B9FFFF MOVZX EAX,WORD PTR SS:[EBP+FFFFB954]
00AE8EF3    8985 C09BFFFF   MOV DWORD PTR SS:[EBP+FFFF9BC0],EAX
00AE8EF9    EB 0C           JMP SHORT 00AE8F07
00AE8EFB    8D85 50B1FFFF   LEA EAX,DWORD PTR SS:[EBP+FFFFB150]
. . .                       . . .
00AE8F09    FFB5 C09BFFFF   PUSH DWORD PTR SS:[EBP+FFFF9BC0]
00AE8F0F    FFB5 88C3FFFF   PUSH DWORD PTR SS:[EBP-3C78]
00AE8F15    E8 6022FEFF     CALL 00ACB17A
```

El salto buscado está en :00AE8EDF y lo que debemos hacer es nopearlo, si ponemos un BP en el lugar donde guarda y otro en el salto comprobaremos que es el mágico y cuando no salta guarda una API buena, lo nopeamos quitando el BP que teníamos en el salto pero dejando el que tenemos donde guarda así vamos viendo como rellena la tabla, no está dejando ningún puntero malo, reiniciamos Olly y llegamos al salto, antes le ponemos un HE para llegar fácil, lonopeamos de entrada para que no redirija ninguna entrada, antes de acabar de rellenarla entera saltan las otras dos llamadas a OutputDebug, esto no sé si está pensado contra un posible script ya que rompe el flujo del bucle, nosotros como lo estamos haciendo con relativa tranquilidad no tenemos problema, superamos ambas llamadas situando el EIP en el RET y seguimos a lo nuestro, cuando vemos que está llegando al final ponemos el BPM en la primera sección, de todos modos siempre es bueno poner un HE en el OEP para estos casos que pare allí sin el BPM, yo lo tengo puesto si alguien no pues que ponga el BPM y ya está igual parará, si vamos al Import ahora a mirar la tabla encontramos que dejó, no olvidar hacer Undo Selection en el salto mágico justo antes de que termine de rellenar la tabla sinó no llegará al OEP.

Venga vamos al Import, seleccionamos el proceso de la lista y...



Ya empezamos con los problemas típicos de Armadillo, os explico: esto tiene muy fácil solución, o bien cogiendo y pegando la cabecera del original (primeros 1000h bytes) o bien reparándolo a mano, yo prefiero de este segundo modo ya que es más entretenido y se aprende más, el motivo de que diga esto es el siguiente:

003F0000	0000C000				Priv	RW	RW
00400000	00001000	FileReco			Imag	R	RWE
00401000	00020000	FileReco	.text		Imag	R	RWE
00421000	00007000	FileReco	.rdata		Imag	R	RWE
00428000	00001000	FileReco	.data		Imag	R	RWE
00429000	00040000	FileReco	.text1	code	Imag	R	RWE
00469000	00010000	FileReco	.adata	SFX	Imag	R	RWE
00479000	00020000	FileReco	.data1	data,import	Imag	R	RWE
00499000	00040000	FileReco	.pdata		Imag	R	RWE
004D9000	00007000	FileReco	.rsrc	resources	Imag	R	RWE
004E0000	00006000				Map	R E	R E

Falta el identificador de la cabecera que es la cadena "PE header" entonces el Import no la reconoce y nos da el error, esto también nos perjudica a la hora del dumpeo ya que en la cabecera está toda la información del exe y no la reconoce. La causa está en que Armadillo nos ha modificado el "PE Signature" que no es más que un puntero al PE real digamos, o sea, donde comienzan los datos en sí, podemos encontrar esta signature en el offet 0x3Ch en todos los casos, es así por convenio, allí debe haber la dirección del PE repito pero en offset, si lo tenemos en :400100 allí encontraremos un 0x100h, veamos este caso concreto:

Address	Hex dump	ASCII
00400000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZé.♥...♦... ..
00400010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	¶.....@.....
00400020	00 00 6A 43 08 00 00 00 00 00 00 00 00 00 00 00	..jC◘.....
00400030	00 00 00 00 00 00 00 00 00 00 00 00 1C F8 0B 00	.....L°♂.
00400040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	♫▼‖♫.┤.=↑@@L=↑Th
00400050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
00400060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS

Estamos en el offset 0x3Ch donde está la Signature, allí el valor que encontramos es uno muy alejado, concretamente :0BF81C, como se el offset representa que el PE debería estar en:

0BF81C + 400000 = 4BF81C

Si vamos a la dirección :4BF81C encontramos un falso PE ya que el nuestro miremos donde está:

Address	Hex dump	ASCII
00400100	50 45 00 00 4C 01 0C 00 41 47 D3 42 00 00 00 00	PE..L♀..AGëB....
00400110	00 00 00 00 E0 00 0F 01 0B 01 53 52 00 E0 03 00	....ó.♫☺♂♪SR.ó♥.
00400120	00 E0 04 00 00 00 00 00 09 90 06 00 00 90 02 00	.ó♦.....ëÅ..ëÅ.
00400130	00 90 07 00 00 00 40 00 00 10 00 00 00 10 00 00	.ë·...@..►...►..

pues si le restamos el imagebase tenemos:

400100 - 400000 = 100

Este es el valor que debería haber en la Signature, lo ponemos:

Address	Hex dump	ASCII
00400000	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00	MZé.♥...♦... ..
00400010	B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00	¶.....@.....
00400020	00 00 6A 43 08 00 00 00 00 00 00 00 00 00 00 00	..jC◘.....
00400030	00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00	.....☺..
00400040	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68	♫▼‖♫.┤.=↑@@L=↑Th
00400050	69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F	is program canno
00400060	74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20	t be run in DOS

Ahora que lo hemos reparado si miramos en la ventana de memoria (Alt+M) veremos:

003F0000	0000C000				Priv	RW	RW
00400000	00001000	FileReco		PE header	Imag	R	RWE
00401000	00020000	FileReco	.text		Imag	R	RWE
00421000	00007000	FileReco	.rdata		Imag	R	RWE
00428000	00001000	FileReco	.data		Imag	R	RWE
00429000	00040000	FileReco	.text1	code	Imag	R	RWE
00469000	00010000	FileReco	.adata	SFX	Imag	R	RWE
00479000	00020000	FileReco	.data1	data,import	Imag	R	RWE
00499000	00040000	FileReco	.pdata		Imag	R	RWE
004D9000	00007000	FileReco	.rsrc	resources	Imag	R	RWE
004E0000	00006000				Map	R E	R E

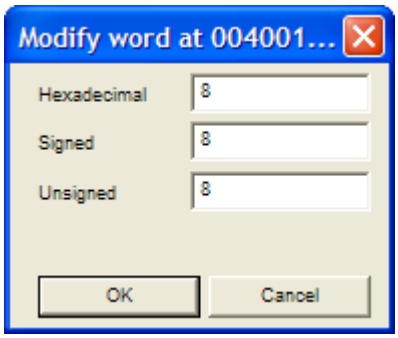
que volvió a aparecer la cadena "PE header" esto es porque ahora sí reconoce la cabecera del archivo como tal, pero aún es pronto para cantar el alirón porque nos hizo una jugarreta más:

```
00400100   50 45 00 0( ASCII "PE"        PE signature (PE)
00400104   4C01       DW 014C            Machine = IMAGE_FILE_MACHINE_I386
00400106   0C00       DW 000C             NumberOfSections = C (12.)
00400108   4147D342   DD 42D34741         TimeDateStamp = 42D34741
0040010C   00000000   DD 00000000         PointerToSymbolTable = 0
00400110   00000000   DD 00000000         NumberOfSymbols = 0
00400114   E000       DW 00E0            SizeOfOptionalHeader = E0 (224.)
```

Nos dice que el número de secciones es 12 (aquí está en formato decimal) si las miramos vemos que las últimas tienen muy mala pinata:

```
00400310   2E 72 73 7( ASCII ".rsrc"     SECTION
00400318   00700000   DD 00007000         VirtualSize = 7000 (28672.)
0040031C   00900D00   DD 000D9000         VirtualAddress = D9000
00400320   00700000   DD 00007000         SizeOfRawData = 7000 (28672.)
00400324   00600800   DD 00086000         PointerToRawData = 86000
00400328   00000000   DD 00000000         PointerToRelocations = 0
0040032C   00000000   DD 00000000         PointerToLineNumbers = 0
00400330   0000       DW 0000             NumberOfRelocations = 0
00400332   0000       DW 0000             NumberOfLineNumbers = 0
00400334   40000040   DD 40000040         Characteristics = INITIALIZED_DATA|READ
00400338   00 00 00 0( ASCII ""           SECTION
00400340   00000000   DD 00000000         VirtualSize = 0
00400344   00000000   DD 00000000         VirtualAddress = 0
00400348   00000000   DD 00000000         SizeOfRawData = 0
0040034C   00000000   DD 00000000         PointerToRawData = 0
00400350   00000000   DD 00000000         PointerToRelocations = 0
00400354   00000000   DD 00000000         PointerToLineNumbers = 0
00400358   0000       DW 0000             NumberOfRelocations = 0
0040035A   0000       DW 0000             NumberOfLineNumbers = 0
0040035C   00000000   DD 00000000         Characteristics = 0
00400360   00 00 00 0( ASCII ""           SECTION
00400368   00000000   DD 00000000         VirtualSize = 0
```

Esta que marqué junto con otras tres son FALSAS secciones, es decir, que el Arma el cabronazo nos las ha añadido por la cara cuando en realidad no existen, sinó de qué esos valores a 0 todoss!! Es una desesperada artimaña más que utiliza, en este caso es la última que le queda, lo reparamos cambiando el valor del número de secciones, con el botón derecho del ratón nos ponemos encima yu elejimos "Modify Integer" esta ventanita se desplegará:



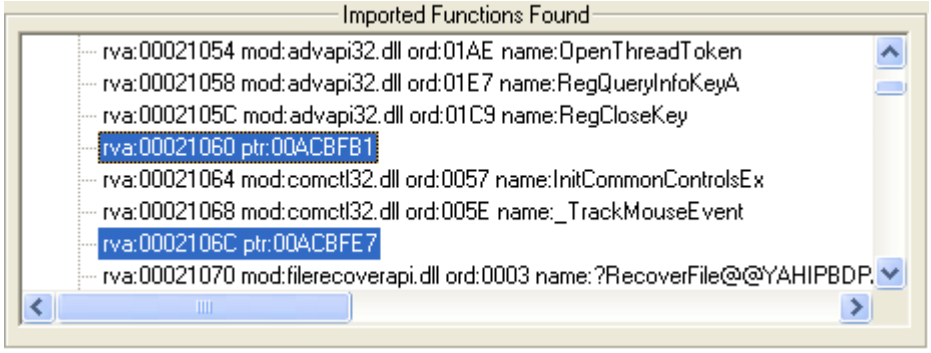
Nada, lo que hacemos es simplemente cambiar el valor "12" que el Arma nos puso allí por un "8" que es el número de secciones reales que tiene nuestro archivo, cuando lo tengamos como en la imagen damos a OK y ya lo tenemos reparado ;-)

Después de este último truquito y una vez reparado podemos ahora sí dumpear, para esto podemos utilizar el plugin IsDebuggerPresent que va muy bien (tipo Icedump), también LordPE nos deja un ejecutable funcional o incluso el Ollydump, esto no necesita más explicación creo ya, lo llamaremos dump.exe para referirnos todos al mismo archivo.

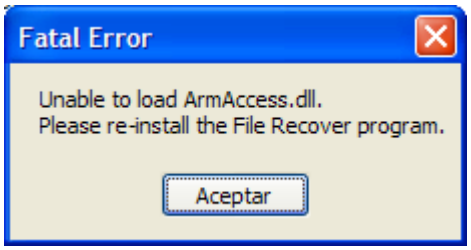
Y ahora que está la cabecera reparada sí podemos ir al Import, le damos los datos:

Notad que nos referimos a las direcciones con offsets y no directamente, le restamos el imagebase del archivo 400000 en este caso (cambia para dll's, etc..)

Cuando lo tengamos le damos a Fix Dump y nos dice que quedaron todavía 13 punteros sin resolver, pero si los miramos más detenidamente nos damos cuenta que son los ceros que deberían ir como separación entre DLL'S y que lógicamente Armadillo no respeta, lo que hace es meter allí valores muy similares a los realmente redirigidos para crear confusión pero no importa a estos les hacemos "Cut Thunks" y quedan listos, veamos como son:

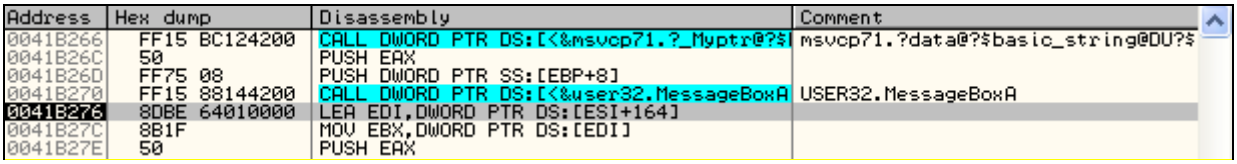


Se ve claramente como son la separación entre librerías, como Import no entiende de esto nos dice que son entradas malas, pero nada como dije antes con un "Cut Thunks" lo resolvemos, le damos luego a Fix Dump y elejimos el dump.exe que antes dejamos, nos dejará un dump_.exe allí en el mismo directorio que es el que deberemos probar, lo ejecutamos y...



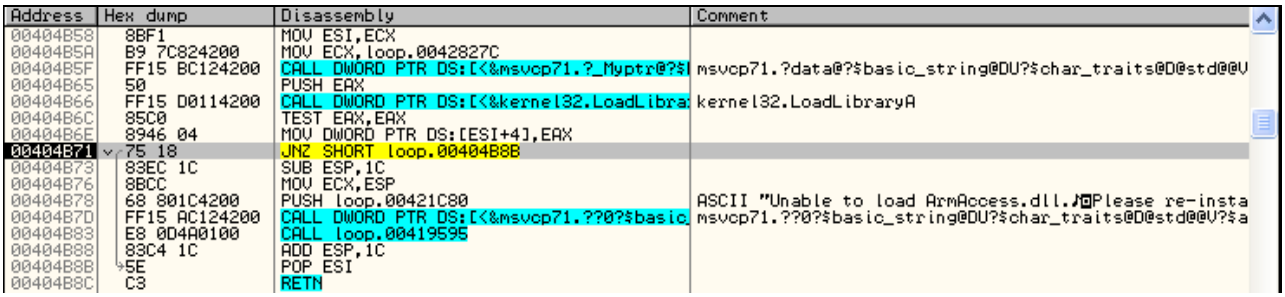
Que nos falta la ArmAccess.dll dice, que es una librería propia de Armadillo, justamente la que lleva el tema de las limitaciones...

Nada, carguemos el dumpeado en Olly… pues no podemos porque pega un petardazo de escándalo que al aceptarlo Olly se cierra y todo ¿? Que está pasando aquí, por mucho que lo intenté no lograba que cargara el archivo, nada se me ocurrió attachearlo de momento justo en el momento que estaba el mensaje de error en pantalla, ya en Olly ir al OEP y hacer un loop infinito allí para luego salvar este archivo como loop.exe, con él haría las pruebas pertinentes sin más coplicaciones que ejecutarlo desde el explorer y attachearlo desde Olly después, quietarlo el loop infinito y a rular, encontré esto:



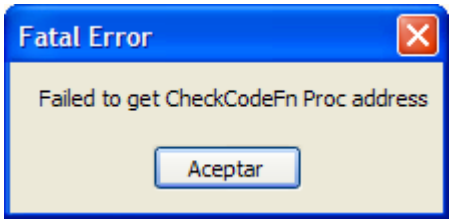
Address	Hex dump	Disassembly	Comment
0041B266	FF15 BC124200	CALL DWORD PTR DS:[<&msvcp71.?_Myptr@?$	msvcp71.?data@?$basic_string@DU?$
0041B26C	50	PUSH EAX	
0041B26D	FF75 08	PUSH DWORD PTR SS:[EBP+8]	
0041B270	FF15 88144200	CALL DWORD PTR DS:[<&user32.MessageBoxA	USER32.MessageBoxA
0041B276	8DBE 64010000	LEA EDI,DWORD PTR DS:[ESI+164]	
0041B27C	8B1F	MOV EBX,DWORD PTR DS:[EDI]	
0041B27E	50	PUSH EAX	

Este es el lugar donde saca el mensaje de error, taceamos un poco a lo retro y vemos:



Address	Hex dump	Disassembly	Comment
00404B58	8BF1	MOV ESI,ECX	
00404B5A	B9 7C824200	MOV ECX,loop.0042827C	
00404B5F	FF15 BC124200	CALL DWORD PTR DS:[<&msvcp71.?_Myptr@?$	msvcp71.?data@?$basic_string@DU?$char_traits@D@std@@U
00404B65	50	PUSH EAX	
00404B66	FF15 D0114200	CALL DWORD PTR DS:[<&kernel32.LoadLibra	kernel32.LoadLibraryA
00404B6C	85C0	TEST EAX,EAX	
00404B6E	8946 04	MOV DWORD PTR DS:[ESI+4],EAX	
00404B71	75 18	JNZ SHORT loop.00404B8B	
00404B73	83EC 1C	SUB ESP,1C	
00404B76	8BCC	MOV ECX,ESP	
00404B78	68 801C4200	PUSH loop.00421C80	ASCII "Unable to load ArmAccess.dll.♪◙Please re-insta
00404B7D	FF15 AC124200	CALL DWORD PTR DS:[<&msvcp71.??0?$basic_	msvcp71.??0?$basic_string@DU?$char_traits@D@std@@U?$a
00404B83	E8 0D4A0100	CALL loop.00419595	
00404B88	83C4 1C	ADD ESP,1C	
00404B8B	5E	POP ESI	
00404B8C	C3	RETN	

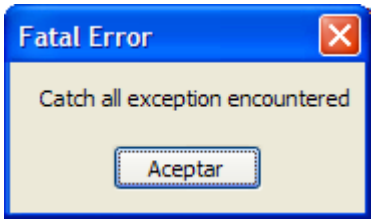
Intenta cargar la libraría ArmAccess.dll que no encuentra y muestra el mensaje de error, si jumpeamos el salto y damos RUN aparece este otro mensaje:



Esto ocurre porque intenta llamar a la función de ArmAccess que cree haber encontrado, pero no fue así ya que modificamos el salto, encontramos un CALL EAX donde falla.

Evitamos entonces el call de alguna manera para que no falle más en él y poder continuar…

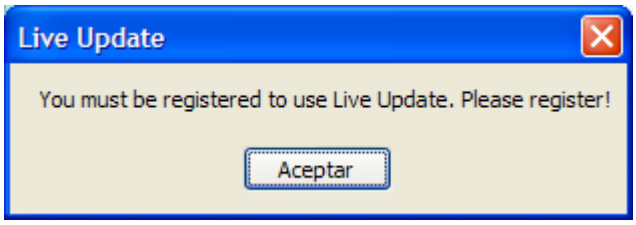
Hecho esto suponemos que los problemas se acabaron verdad? nadamás lejos de la realidad, volvemos a intentarlo pero el muy mamón vuelve a fallar, esta vez se queja de esta manera:



Pues parece que no hay manera, encontramos el código y todo donde sucede el error pero suponemos que seguir por esta linea es algo complejo por el tema de las falsas llamadas y eso, cada ver que llama a GetProcAddress guarda el valor de retorno (dirección de la función buscada) como le engañamos a la hora de llamarla falla ya que no encontró nada...

Aquí la solución fácil pasa por coger una librería ArmAccess.dll de otro Armadillo y ponérsela en su misma carpeta y así la encuentre, veremos que así sí que corre, bueno algo es algo, ya conseguimos desempakarlo y ahora que corra, bouuuuff!!

Vayamos pues a por las limitaciones que tiene, la primera, si intentamos abrir el LiveUpdate nos encontramos este cartelito que nos mete un broncón, jajaja:



O sea que esta función está reservada para usuarios registrados, y como nosotros todavía no lo estamos ;-DDD no la podemos utilizar…

Pues vaya por Dios, nos lo compramos? Jeje.. Después además nos abre una nag para que nos regisrtemos y todo, pero nada…

Visto el mensaje nos abre una puerta para el ataque, si no dijera nada no pero aquí hay un mensaje y un millón de API's eperando a ser llamadas y parar la ejecución, vamos a ello, antes de picar en Live Update ponemos un precioso BP en messageBoxA y luego si picamos en la función prohibida, Olly no para y aparece directamente la segunda nag, jajaja.. ha detectado el BP, lo ponemos pero HE esta vez, y tampoco para, jooooooder como se las gasta, en cualquier caso saca la segunda nag que es como un banner, podemos usar un BP en DestroyWindow para pillar el momento donde la destruye y así llegar al lugar adecuado que es este:

```
00AD01C2    FF75 18         PUSH DWORD PTR SS:[EBP+18]
00AD01C5    FF75 14         PUSH DWORD PTR SS:[EBP+14]
00AD01C8    FF75 10         PUSH DWORD PTR SS:[EBP+10]
00AD01CB    FF75 0C         PUSH DWORD PTR SS:[EBP+C]
00AD01CE    FF75 08         PUSH DWORD PTR SS:[EBP+8]
00AD01D1    FF15 3C24AF00   CALL DWORD PTR DS:[AF243C]              ; USER32.DialogBoxParamA
```

Es la zona donde se crea la segunda nag, nos acercamos, seguimos superando algunos RET's hsta que por fin llegamos a este mgnífico lugar:

```
004151DD    81F9 69040000   CMP ECX,469
004151E3    75 0F           JNZ SHORT FileReco.004151F4    ;Si salta hace la función
004151E5    8BCB            MOV ECX,EBX
004151E7    8975 FC         MOV DWORD PTR SS:[EBP-4],ESI
004151EA    E8 02E3FFFF     CALL FileReco.004134F1         ;Call restrictor
004151EF    E9 8F000000     JMP FileReco.00415283
```

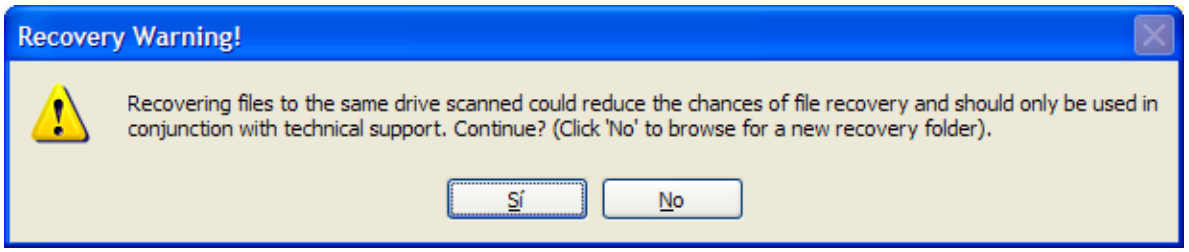
Como vemos en los comentarios la restricción "se oculta" en el CALL después hay un JMP que nos envía a freír espárragos, o sea, bien lejos de la función solicitada, peeeeeeero antes vemos un JNZ que de producirse tocamos el CIELO, probemos a jumpearlo y...

Se reinicia el programa ¿? Esto será normal? Bueno quizá elegimos la función inadecuada al caso, nada vamos por otra limitación, en Advanced Scan hacemos un scanneo rápido pues lo cancelamos antes para no perder tiempo, algunos archivos ya están allí, seleccionamos uno cualquiera y le damos a Recover, jejeje.. el mismo mensaje de siempre, ponemos un HE en MessageBoxA que creí que detectaba pero no es que se me olvidó quitar el BP antes, bueno siguiendo el mensaje llegamos a esta parte:

```
0040CE14    8B4D BC         MOV ECX,DWORD PTR SS:[EBP-44]
0040CE17    2B4D B8         SUB ECX,DWORD PTR SS:[EBP-48]
0040CE1A    C1F9 02         SAR ECX,2
0040CE1D    03CF            ADD ECX,EDI
0040CE1F    3BC8            CMP ECX,EAX
0040CE21    8BCB            MOV ECX,EBX
0040CE23    7E 0A           JLE SHORT IsDebug_.0040CE2F    ;Si salta hace la función
0040CE25    E8 45F2FFFF     CALL IsDebug_.0040C06F         ;Call restrictor
0040CE2A    E9 20010000     JMP IsDebug_.0040CF4F
```

Es donde decide si realizar o no la función de Recuperar los archivos, al menos esto es lo que parece, sobretodo por ese JMP que nos envía bastante lejos y el JLE anterior que si se produce aparte de evitar el call restrictor os mete un poquitín más abajo del JMP donde supuestamente se realice la función.

Probemos esta hipótesis, demos nuevamente a Recover, pero antes ponemos un BP en :40CE1F, cuando Olly pare cambiamos el valor a EAX==2 para que sea mayor que ECX en la comparación y salte despúés en el JLE que vien, hecho esto aparece este mensaje:



jajajaja... nos pregunta si queremos continuar, nos informa además de algunas cosillas, nada aceptamos por supuesto, se pone a trabajar como loco, SIIIIIII..!! hace la función que le pedimos sin rechistar, es de suponer que íbamos por el buen camino antes en el LiveUpdate, bueno aún quedan un par de temas, intentamos pulsar el botón "Browse" y lo mismo, saltan los BP's que tenemos puestos y rápido llegamos al lugar

```
004095F6    E8 58B5FFFF     CALL IsDebug_.00404B53
004095FB    84C0            TEST AL,AL
004095FD    75 0E           JNZ SHORT IsDebug_.0040960D    ;Si salta hace la función
004095FF    8BCE            MOV ECX,ESI
00409601    E8 692A0000     CALL IsDebug_.0040C06F         ;Call restrictor
00409606    32C0            XOR AL,AL
00409608    E9 9A000000     JMP IsDebug_.004096A7
```

Vaya, vaya, esta vez el salto va precedido de un call, miremos donde apunta:

```
00404B53    8A41 44         MOV AL,BYTE PTR DS:[ECX+44]
00404B56    C3              RETN
```

En el momento de la llamada [ECX+44] representa [428264] y no cal ni tocarlo ya que cambiando:

```
00404B53    8A41 44         MOV AL,BYTE PTR DS:[ECX+44]    POR:
00404B53    B001            MOV AL, 01
00404B55    90              NOP
```

Hace la función sin problemas, vemos que además es el complemento ideal al cambio:

```
0040CE23    7E 0A           JLE SHORT IsDebug_.0040CE2F    POR:
0040CE23    EB 0A           JMP SHORT IsDebug_.0040CE2F
```

Ya que con estos cambios el programa está en nuestras manos y hace todo lo que queramos.

Para que ponga nuestro nombre en el About y en el diálogo principal cambiamos:

```
            0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
000215a0h: FF FF FF FF 2E 10 40 00 3C 10 40 00 53 79 58 65 ; ÿÿÿÿ..@.<.@.SyXe
000215b0h: 27 30 35 00 00 00 00 00 00 00 00 00 55 53 45 52 ; '05.........USER
000215c0h: 4E 41 4D 45 00 00 00 00 55 53 45 52 4B 45 59 00 ; NAME....USERKEY.
000215d0h: 52 45 47 4C 49 4D 5F 4D 41 58 46 49 4C 45 53 00 ; REGLIM MAXFILES.
```

Lo podemos hacer con UltraEdit como en la imagen, la cuestión es poner allí nuestro nombre y tenemos el programa totalmente crackeado. Como observación decir que además el botón que era "Boy Now" ha cambiado a "About" lo que indica que hicimos bien el trabajo.

Y con esto se termina el tutorial, espero que haya sido de gran ayuda a los que empiezan ahora, para ellos y por ellos escribo estos textos, si necesitan cualquier cosa me pueden escribir tranquilamente a estas direcciones:

syxe00@yahoo.es
syxe00@hotmail.com

preferiría que me escribiérais a la de yahoo…

Un saludo a todo CracksLatinos, dedicado a todos ellos, en especial a Ricardo, +NCR, [ZekkT] y todo aquél que colabore con sus tuts, cursos y escribe lo que aprende.! Hasta la próxima.!